claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a6b422b95bbb6eaaf.jsonl`
- SHA-256: `a41d32c98bfb8a4c9a0ba5fb455fdd92cc2bb90c8f9a367177e4a042741eeb14`
- Source modified: `2026-06-13T19:16:33+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:15:00.403Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"PersonDetector.detections_per_frame (the new GPU-adjacent method) has zero test coverage despite its loop being fully testable no-GPU","severity":"high","file":"backend/pipeline/detectors/person_detector.py:255-301 ; tests/test_fusion_hybrid.py:99-109","detail":"detections_per_frame is the only NEW non-pure code added, and it contains exactly the kind of off-by-one-prone logic that should be unit-tested: absolute-frame seek (cap.set(CAP_PROP_POS_FRAMES, start_frame)), the sampling stride `(frame_idx - start_frame) % step == 0`, the `while frame_idx <= end_frame` boundary (note: inclusive end + the modulo is relative to start_frame, not absolute frame index), and the fps<=0 -> 30.0 / not-opened fallbacks. The fusion-hybrid test mocks the WHOLE method out (fake_pd.detections_per_frame.return_value = {...}) and only asserts `assert_called()`, so none of this logic runs. This is NOT a genuine GPU gap: tests/test_person_detector.py already proves the exact harness (mock cv2.VideoCapture via _make_cap_mock with cap.read.side_effect, mock pd.model/make_tracker/detect_and_track) that drives this loop entirely on CPU. The frame-index alignment that fusion_features.shuttle_player_colocation / two_sided_motion depend on (per the method's own docstring) is asserted nowhere.","fix":"Add a test in test_person_detector.py mirroring test_extract_action_windows_returns_enriched_dicts: patch cv2.VideoCapture with a cap mock, set detect_and_track side_effect, call detections_per_frame(start_frame=30, end_frame=40, frame_skip=3) and assert the returned dict has frames keyed at exactly {30,33,36,39}, that frame_width/frame_height/fps come from the cap, and that frames beyond end_frame / before start_frame are absent. Also cover the not-opened branch (returns frame_width=0.0, frame_height=0.0)."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:15:03.076Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:15:04.035Z)

```
240             windows = []
241             group = [active_frames[0]]
242             for af in active_frames[1:]:
243                 if af[0] - group[-1][0] <= max_gap_frames:
244                     group.append(af)
245                 else:
246                     windows.append(_finalize(group))
247                     group = [af]
248             windows.append(_finalize(group))
249 
250             # Filter out extremely short action windows
251             min_window_duration = self.config.get("indexing",
{}).get("min_action_window_duration", 2.0)
252             windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
253 
254             return windows
255 
256         def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
257                                  frame_skip: int = 3) -> Dict[str, Any]:
258             """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
video.
259 
260             Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
261             ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
discards the
262             per-frame boxes), this returns the raw per-frame detections so the fusion
features can
263             align them with the shuttle trajectory. It reads the source by **absolute frame
index**
264             (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
match
265             the WASB trajectory's ``.frame`` exactly ? that alignment is what
266             ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
267 
268             Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
269             "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
seek
270             accuracy varies by codec (it lands near the requested keyframe then decodes
forward);
271             treat the indices as ±a few frames on real footage ? fine for window-level
features.
272             """
273             self.lazy_load_model()
274             cap = cv2.VideoCapture(video_path)
275             if not cap.isOpened():
276                 logger.error(f"Could not open {video_path}")
277                 return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
278 
279             fps = cap.get(cv2.CAP_PROP_FPS)
280             if fps <= 0:
281                 fps = 30.0
282             frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
283             frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
284             score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
285             step = max(frame_skip, 1)
286             tracker = self.make_tracker(fps / step)
287 
288             start_frame = max(0, int(start_frame))
289             frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
290             cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
291             frame_idx = start_frame
292             while frame_idx <= end_frame:
```

```
293                 ret, frame = cap.read()
294                 if not ret:
295                     break
296                 if (frame_idx - start_frame) % step == 0:
297                     frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
298                 frame_idx += 1
299             cap.release()
300             return {"frames": frames, "fps": fps,
301                     "frame_width": float(frame_width), "frame_height": float(frame_height)}
302
```

4. user (2026-06-13T19:15:04.439Z)

```
1       """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3       Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4       keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5       substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6       persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7       """
8       from types import SimpleNamespace
9       from unittest.mock import MagicMock, patch
10
11      from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12      from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14      # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15      # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16      WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17               "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18      WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19               "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22      def _points():
23          pts = []
24          for f in range(30, 120):       # window A frames: y flips 100<->900 around the net
(~500)
25              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
26          for f in range(300, 360):      # window B frames: y == net level -> deadband -> no
crossings
27              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
28          return pts
29
30
31      def _runner():
32          r = MagicMock()
33          r.healthcheck.return_value = (True, "env ok")
34          r.run_predict.return_value = "out.csv"
35          r.parse_trajectory_csv.return_value = _points()
36          r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
37          return r
38
39
40      def _run(indexing=None, provider_segments=None):
41          provider = MagicMock()
42          provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
43          db = MagicMock()
44          db.add_candidate_segment.side_effect = [101, 102, 103, 104]
45          db.add_play_segment.return_value = 200
```

```
46
47          with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
48                patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)),
\
49                patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
50                patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
51                patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
52                patch("os.environ.get", return_value="dummy_key"):
53               preg.get.return_value = provider
54               seg = FusionSegmenter(db, {"indexing": indexing or {}})
55               res = seg.process_video("v1", "clip.mp4")
56               return db, provider, res
57
58
59      def _stats_calls(db):
60          """Persisted stats dicts, in call order."""
61          return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
62
63
64      def test_net_crossings_persisted_and_person_off_by_default():
65          db, _provider, _res = _run()
66          stats = _stats_calls(db)
67          assert len(stats) == 2                              # both windows persisted
(superset)
68          assert all("net_crossings" in s for s in stats)
69          # Window A (rally) crosses many times; window B (held) zero.
70          assert stats[0]["net_crossings"] >= 2
71          assert stats[1]["net_crossings"] == 0.0
72          # Person cue OFF by default -> NONE of the person features present.
73          assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
74          # The 4 base motion keys are still there.
75          assert all({"peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count"} <= set(s)
76                     for s in stats)
77
78
79      def test_persisted_candidates_are_superset_of_substrate_windows():
80          db, _provider, _res = _run()
81          spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
db.add_candidate_segment.call_args_list]
82          assert spans == [(1.0, 4.0), (10.0, 12.0)]          # exactly the substrate windows,
unfiltered
83
84
85      def test_default_hands_off_all_windows():
86          _db, provider, _res = _run()                        # min_crossings default 0 -> no
gating
87          assert provider.analyze_video.call_count == 2
88
89
90      def test_gate_a_drops_low_crossing_window_from_handoff_only():
91          db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
92          # Persistence is still the full superset (offline substrate intact)...
93          assert len(_stats_calls(db)) == 2
94          # ...but only the high-crossing window A reaches the AI handoff.
95          assert provider.analyze_video.call_count == 1
96
97
98      def test_person_cue_adds_features_when_enabled():
99          fake_pd = MagicMock()
100         fake_pd.detections_per_frame.return_value = {
```

```
101             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},
102             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
103         }
104         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
return_value=fake_pd):
105             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
106         stats = _stats_calls(db)
107         assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
108         assert all("net_crossings" in s for s in stats)
109         fake_pd.detections_per_frame.assert_called()         # the person detector WAS used
110
```

5. assistant (2026-06-13T19:15:09.146Z)

Now let me verify the claims in the finding. Let me check the test_person_detector.py file to confirm the existing harness exists and whether `detections_per_frame` has any coverage there.

6. user (2026-06-13T19:15:09.876Z)

```
1       """Unit tests for the person-detection rally cue
(backend.pipeline.detectors.person_detector).
2
3       Pure/offline ? the torchvision model and supervision tracker are mocked, so these run
4       without GPU/torch weights. They pin the COCO person==1 filter (the YOLO-was-0 gotcha)
and
5       the motion-window extraction that feeds the candidate `stats` + the learned fusion
features.
6       """
7       from unittest.mock import MagicMock, patch
8
9       from backend.pipeline.detectors.person_detector import PersonDetector, _PERSON_CLASS_ID
10
11
12      def _make_cap_mock(num_frames=300):
13          """A cv2.VideoCapture mock yielding `num_frames` readable frames at 30fps."""
14          import cv2
15          cap_mock = MagicMock()
16          cap_mock.isOpened.return_value = True
17
18          def mock_get_prop(prop):
19              if prop == cv2.CAP_PROP_FPS:
20                  return 30.0
21              if prop == cv2.CAP_PROP_FRAME_WIDTH:
22                  return 1000.0
23              return 0.0
24
25          cap_mock.get.side_effect = mock_get_prop
26          cap_mock.read.side_effect = [(True, "frame")] * num_frames + [(False, None)]
27          return cap_mock
28
29
30      def _make_detections(num_frames=300, step=5, track_id=1):
31          """One steadily-moving person per frame (box advances `step` px/frame)."""
32          return [[(track_id, (float(i * step), 0.0, float(i * step + 10), 10.0))]
33                  for i in range(num_frames)]
34
35
36      @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
37      def test_extract_action_windows_returns_enriched_dicts(mock_cap):
38          pd = PersonDetector({"indexing": {"min_action_window_duration": 1.0}})
39          pd.model = MagicMock()                                  # short-circuits
lazy_load_model
40          pd.make_tracker = MagicMock(return_value=MagicMock())
41          pd.detect_and_track = MagicMock(side_effect=_make_detections())
42          mock_cap.return_value = _make_cap_mock()
```

```
43
44          windows = pd.extract_action_windows_from_chunk(
45              "chunk.mp4", chunk_start=5.0, velocity_thresh=0.05, merge_gap=2.0,
46              frame_skip=1, chunk_index=2,
47          )
48
49          assert len(windows) >= 1
50          w = windows[0]
51          assert set(w) >= {"start", "end", "active_frame_count", "peak_velocity",
52                            "mean_velocity", "track_id_count", "chunk_index"}
53          assert w["chunk_index"] == 2
54          assert w["start"] >= 5.0                                # full-video coordinates
55          assert w["active_frame_count"] > 0
56          assert w["peak_velocity"] >= w["mean_velocity"] > 0
57          assert w["track_id_count"] == 1
58
59
60      def test_detect_and_track_filters_persons_and_returns_track_ids():
61          """detect_and_track keeps only confident PERSON detections, feeds them to the
62          tracker, and returns (track_id, bbox) tuples ? exercising the real torchvision
63          output-shape handling and the COCO person==1 filter (the YOLO-was-0 gotcha)."""
64          import numpy as np
65
66          pd = PersonDetector({"indexing": {}})
67          pd.device = "cpu"
68
69          # Fake torchvision detector output: a person (kept), a low-score person (dropped),
70          # and a racket/other class (dropped).
71          out = {"boxes": MagicMock(), "labels": MagicMock(), "scores": MagicMock()}
72          out["boxes"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
73              [[0, 0, 10, 10], [5, 5, 15, 15], [20, 20, 30, 30]], dtype=float)
74          out["labels"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
75              [_PERSON_CLASS_ID, _PERSON_CLASS_ID, 38])           # 38 = sports-ball-ish, not
person
76          out["scores"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
77              [0.9, 0.1, 0.99])
78          pd.model = MagicMock(return_value=[out])
79
80          # Fake tracker echoes the detections it receives, assigning a single track id.
81          tracker = MagicMock()
82          tracked = MagicMock()
83          tracked.__len__.return_value = 1
84          tracked.tracker_id = [7]
85          tracked.xyxy = np.array([[0, 0, 10, 10]], dtype=float)
86          tracker.update_with_detections.return_value = tracked
87
88          with patch("backend.pipeline.detectors.person_detector.cv2.cvtColor",
return_value=np.zeros((10, 10, 3))), \
89               patch("backend.pipeline.detectors.person_detector.torch") as mock_torch, \
90               patch.dict("sys.modules", {"supervision": MagicMock()}):
91              mock_torch.no_grad.return_value.__enter__ = lambda s: None
92              mock_torch.no_grad.return_value.__exit__ = lambda s, *a: None
93              # torch.from_numpy(...).permute(...).float().div(...).to(...) -> dummy tensor
94              mock_torch.from_numpy.return_value.permute.return_value.float.return_value.div.r
eturn_value.to.return_value = "tensor"
95              result = pd.detect_and_track("frame", tracker, score_thresh=0.5)
96
97          # Only the high-confidence person survived the score+class filter
98          assert result == [(7, (0.0, 0.0, 10.0, 10.0))]
99          # And exactly that filtered set (1 box) was handed to the tracker
100         tracker.update_with_detections.assert_called_once()
101
```

7. assistant (2026-06-13T19:15:14.978Z)

The finding is verified so far. `test_person_detector.py` has tests for `extract_action_windows_from_chunk` and `detect_and_track`, but no test for `detections_per_frame`. Let me confirm by searching the entire test suite for any reference to `detections_per_frame`.

8. user (2026-06-13T19:15:16.289Z)

```
backend\pipeline\detectors\person_detector.py:256:    def detections_per_frame(self, video_path:
str, start_frame: int, end_frame: int,
backend\pipeline\segmenters\fusion_hybrid.py:98:        det =
self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
tests\test_fusion_hybrid.py:100:    fake_pd.detections_per_frame.return_value = {
tests\test_fusion_hybrid.py:109:    fake_pd.detections_per_frame.assert_called()         # the
person detector WAS used
```

9. assistant (2026-06-13T19:15:19.412Z)

Confirmed. The only test references to `detections_per_frame` are in `test_fusion_hybrid.py`, where it's fully mocked (`return_value` set, `assert_called()`). No test exercises the actual loop. Let me verify the claim that `detections_per_frame` is genuinely the only NEW non-pure code added in this change, and check the git diff to confirm the method is new (not pre-existing).

10. user (2026-06-13T19:15:20.619Z)

```
diff --git a/backend/pipeline/detectors/person_detector.py
b/backend/pipeline/detectors/person_detector.py
index 38ec2b3..5b931a4 100644
--- a/backend/pipeline/detectors/person_detector.py
+++ b/backend/pipeline/detectors/person_detector.py
@@ -252,3 +252,50 @@ class PersonDetector:
         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

         return windows
+
+    def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
+                             frame_skip: int = 3) -> Dict[str, Any]:
+        """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL video.
+
+        Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
+        ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and discards the
+        per-frame boxes), this returns the raw per-frame detections so the fusion features can
+        align them with the shuttle trajectory. It reads the source by **absolute frame index**
+        (seek + sequential read), NOT a re-extracted clip, so the returned frame indices match
+        the WASB trajectory's ``.frame`` exactly ? that alignment is what
+        ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
+
+        Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
+        "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
+        accuracy varies by codec (it lands near the requested keyframe then decodes forward);
+        treat the indices as ±a few frames on real footage ? fine for window-level features.
+        """
+        self.lazy_load_model()
+        cap = cv2.VideoCapture(video_path)
+        if not cap.isOpened():
+            logger.error(f"Could not open {video_path}")
+            return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
+
+        fps = cap.get(cv2.CAP_PROP_FPS)
+        if fps <= 0:
+            fps = 30.0
+        frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
+        frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
+        score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
```

```
+        step = max(frame_skip, 1)
+        tracker = self.make_tracker(fps / step)
+
+        start_frame = max(0, int(start_frame))
+        frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
+        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
+        frame_idx = start_frame
+        while frame_idx <= end_frame:
+            ret, frame = cap.read()
+            if not ret:
+                break
+            if (frame_idx - start_frame) % step == 0:
+                frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
+            frame_idx += 1
+        cap.release()
+        return {"frames": frames, "fps": fps,
+                "frame_width": float(frame_width), "frame_height": float(frame_height)}
```

11. assistant (2026-06-13T19:15:23.920Z)

Confirmed: the entire `detections_per_frame` method is new code in this change (the diff shows it added as a fresh method after line 252). Now let me verify the claim that this is the "only NEW non-pure code" by checking the diffs of the other edited files, and that the other new files are pure.

12. user (2026-06-13T19:15:24.974Z)

```
warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be
replaced by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_trajectory_hybrid_equivalence.py', LF will be
replaced by CRLF the next time Git touches it
 backend/config/models.py                          | 34 ++++++++++++++++
 backend/pipeline/detectors/person_detector.py     | 47 ++++++++++++++++++++++
 backend/pipeline/segmenters/__init__.py           |  2 +
 backend/pipeline/segmenters/trajectory_hybrid.py | 50 ++++++++++++++++++++++-----
 docs/README.md                                    |  3 +-
 tests/test_config.py                              | 30 ++++++++++++++
 tests/test_trajectory_hybrid_equivalence.py       | 22 ++++++++++
 7 files changed, 177 insertions(+), 11 deletions(-)
---NEW FILES---
 M backend/config/models.py
 M backend/pipeline/detectors/person_detector.py
 M backend/pipeline/segmenters/__init__.py
 M backend/pipeline/segmenters/trajectory_hybrid.py
 M docs/README.md
 M tests/test_config.py
 M tests/test_trajectory_hybrid_equivalence.py
?? artifacts/
?? backend/pipeline/detectors/fusion_features.py
?? backend/pipeline/segmenters/fusion_hybrid.py
?? docs/QUALITY_ITERATIONS.md
?? scratch/HIGH_RISK_REMEDIATION_PLAN.md
?? scratch/PR1_BODY.md
?? scratch/PR1_HANDOFF.md
?? scratch/PR2_HANDOFF.md
?? scratch/PR2_deferred_hardening_status.patch
?? scratch/PR88_FIX.md
?? scratch/arial.ttf
?? scratch/debug_wsl.py
?? scratch/keep_awake.ps1
?? scratch/probe_cv2_count.py
?? scratch/probe_gemini.py
?? scratch/repro_drawtext_font.py
?? scratch/repro_movie_overlay.py
?? scratch/run_gx010125.py
```

```
?? scratch/stitch_min5.py
?? scratch/trim_video.py
?? tests/test_fusion_features.py
?? tests/test_fusion_hybrid.py
```

13. assistant (2026-06-13T19:15:27.819Z)

Let me check the new fusion_features.py to confirm it's pure (and has its own test), and check the fusion_hybrid.py around line 98 to understand how `detections_per_frame` is called and how alignment matters.

14. user (2026-06-13T19:15:29.188Z)

```
75              # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76              # over the whole trajectory by rally_gate (camera-agnostic).
77              gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
78              stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80              fcfg = idx_cfg.get("fusion", {})
81              if fcfg.get("cue_flags", {}).get("person", False):
82                  stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83              return stats
84
85          def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                               frame_width: float, video_path: str,
87                               fcfg: Dict[str, Any]) -> Dict[str, float]:
88              """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89              frame range and compute the scale/translation-invariant person features. Lazily
builds
90              ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91              # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92              from backend.pipeline.detectors.person_detector import PersonDetector
93              if self._person is None:
94                  self._person = PersonDetector(self.config)
95
96              frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97              start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98              det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99              fw = det["frame_width"] or frame_width
100             fh = det["frame_height"] or 0.0
101
102             poly = fcfg.get("court_polygon")
103             polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104             net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105             shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106             return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                               court_polygon=polygon, net=net)
108
109         def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                                 window_stats: List[Dict[str, Any]],
111                                 idx_cfg: Dict[str, Any]) -> List[int]:
112             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113             (0) keep every window ? identical to the substrate. Persisted candidates are
114             untouched (full superset)."""
115             fcfg = idx_cfg.get("fusion", {})
116             min_crossings = int(fcfg.get("min_crossings", 0))
117             min_players = int(fcfg.get("min_players", 0))
118             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
```

```
119
120             keep: List[int] = []
121             for i, st in enumerate(window_stats):
122                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                     continue
124                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
```

15. user (2026-06-13T19:15:30.144Z)

```
warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be
replaced by CRLF the next time Git touches it
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index b2847c9..149282e 100644
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
@@ -78,6 +78,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         except (ValueError, TypeError):
             return max(3, int(duration / 0.8))

+    # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -------
+    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
+                                frame_width: float, video_path: str,
+                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
+        """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
+        returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
+        rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
+        features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
+        return {
+            "peak_velocity": cw["peak_velocity"],
+            "mean_velocity": cw["mean_velocity"],
+            "active_frame_count": cw["active_frame_count"],
+            "track_id_count": cw["track_id_count"],
+        }
+
+    def _select_for_handoff(self, windows: List[Dict[str, Any]],
+                            window_stats: List[Dict[str, Any]],
+                            idx_cfg: Dict[str, Any]) -> List[int]:
+        """Indices of the candidate windows that proceed to the AI handoff (and thus may
+        become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
+        ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
+        Persisted candidates are NOT affected ? they remain the full superset for offline
+        re-scoring."""
+        return list(range(len(windows)))
+
     def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                       player_pool: Optional[List[str]] = None,
                       max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
@@ -172,31 +197,36 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                             f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                             f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

+        # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
+        # segmenter adds net_crossings + person-cue features. Computed once, reused for both
+        # persistence and the handoff gate below.
+        window_stats = [
+            self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
+            for cw in all_candidate_windows
+        ]
+
         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
```

```
         if store_candidates:
             for i, cw in enumerate(all_candidate_windows):
-                stats = {
-                    "peak_velocity": cw["peak_velocity"],
-                    "mean_velocity": cw["mean_velocity"],
-                    "active_frame_count": cw["active_frame_count"],
-                    "track_id_count": cw["track_id_count"],
-                }
                 candidate_ids[i] = self.db.add_candidate_segment(
                     video_id, detector=self.name,
                     start_time=cw["start"], end_time=cw["end"],
                     chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
-                    stats=stats, detection_params=detection_params,
+                    stats=window_stats[i], detection_params=detection_params,
                 )

         if not all_candidate_windows:
             return []

         # --- Phase 3: AI provider handoff over padded candidate clips ---
+        # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
+        # candidates above stay the full superset ? only the served play_segments are gated.
+        handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
         os.makedirs(handoff_dir, exist_ok=True)
         logger.info("Phase 3: %s validation on %d candidate windows...",
-                    provider_name, len(all_candidate_windows))
+                    provider_name, len(handoff_indices))

         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
             w_start = max(0, window["start"] - handoff_padding)
@@ -225,8 +255,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):

         ai_segments: List[dict] = []
         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
-            futures = [executor.submit(process_candidate_window, wi, cw)
-                       for wi, cw in enumerate(all_candidate_windows)]
+            futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
+                       for wi in handoff_indices]
             for future in futures:
                 ai_segments.extend(future.result())
         try:
```

16. assistant (2026-06-13T19:15:34.969Z)

The diff confirms the finding's claim. The edits to `trajectory_hybrid.py` are pure orchestration (hooks returning identity), `config/models.py` is config schema, and `fusion_hybrid.py`/`fusion_features.py` are the new fusion logic. The only NEW code that touches cv2/GPU with non-trivial loop logic is `detections_per_frame`.

Let me verify the key claim about the alignment dependency: `fusion_features` uses the frame indices that `detections_per_frame` produces. Let me confirm `compute_person_features` keys frames the way the method emits them.

17. user (2026-06-13T19:15:35.849Z)

```
1       """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3       NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4       math is unit-testable in a GPU-free dev container. The detection (person boxes,
5       shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6       module only turns those signals into a handful of **scale/translation-invariant**
7       features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
```

```
8       memorise this court/camera and fail to generalise off a small labelled set).
9
10      Conventions:
11      - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12      - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14      - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15      - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17      - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19      Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20      floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21      """
22      from __future__ import annotations
23
24      from typing import Dict, List, Optional, Sequence, Tuple
25
26      BBox = Tuple[float, float, float, float]
27      Net = Tuple[str, float]
28
29
30      def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31          """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32          (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33          if polygon is None or len(polygon) < 3:
34              return False
35          x, y = point
36          inside = False
37          n = len(polygon)
38          j = n - 1
39          for i in range(n):
40              xi, yi = polygon[i]
41              xj, yj = polygon[j]
42              # Does the horizontal ray at y cross edge (i, j)?
43              if (yi > y) != (yj > y):
44                  x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                  if x < x_cross:
46                      inside = not inside
47              j = i
48          return inside
49
50
51      def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52          """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53          the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54          if frame_width <= 0 or frame_height <= 0:
55              return (0.0, 0.0)
56          x1, _y1, x2, y2 = box
57          return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60      def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61          if frame_width <= 0 or frame_height <= 0:
62              return (0.0, 0.0)
63          x1, y1, x2, y2 = box
```

```
        return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)


    def person_in_court(box: BBox, frame_width: float, frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
        """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
        True (count everyone) when it is None."""
        if court_polygon is None:
            return True
        return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)


    def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                        frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
        """Per-frame count of in-court persons (one entry per sampled frame)."""
        return [
            sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
            for _frame_idx, dets in sorted(per_frame.items())
        ]


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))


    def players_in_court(counts: Sequence[int]) -> float:
        """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
        return _median(list(counts))


    def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
        """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
        if not counts:
            return 0.0
        return sum(1 for c in counts if c >= min_players) / len(counts)


    def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
                           frame_width: float, frame_height: float) -> float:
        """Mean per-track centre displacement between consecutive sampled frames, normalised
        by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
        if frame_width <= 0 or len(per_frame) < 2:
            return 0.0
        frames = sorted(per_frame.keys())
        steps: List[float] = []
        for a, b in zip(frames, frames[1:]):
            prev = {tid: box for tid, box in per_frame[a]}
            for tid, box in per_frame[b]:
                if tid in prev:
                    cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
                    cx1, cy1 = _center_norm(box, frame_width, frame_height)
                    steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
        if not steps:
            return 0.0
        return sum(steps) / len(steps)


```

```
    def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                         frame_height: float, net: Optional[Net],
                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
        """Fraction of sampled frames with >=1 in-court person on BOTH net halves.

        A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
        The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
        """
        if net is None or not per_frame:
            return 0.0
        axis, pos = net
        both = 0
        for _frame_idx, dets in per_frame.items():
            near, far = False, False
            for _tid, box in dets:
                if not person_in_court(box, frame_width, frame_height, court_polygon):
                    continue
                fx, fy = _foot_norm(box, frame_width, frame_height)
                coord = fx if axis == "x" else fy
                if coord < pos:
                    near = True
                else:
                    far = True
            if near and far:
                both += 1
        return both / len(per_frame)


    def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
                        pad_frac: float) -> bool:
        """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
        width/height (the 'adjacent / in-hand' tolerance)?"""
        x1, y1, x2, y2 = box
        px = pad_frac * frame_width
        py = pad_frac * frame_height
        return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)


    def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
                                  frame_width: float, frame_height: float,
                                  pad_frac: float = 0.03) -> float:
        """Fraction of (visible shuttle frames that were also person-sampled) in which the
        shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
        ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
        """
        if not per_frame or frame_width <= 0 or frame_height <= 0:
            return 0.0
        aligned = 0
        colocated = 0
        for p in shuttle_points or []:
            if not getattr(p, "visible", False):
                continue
            dets = per_frame.get(int(p.frame))
            if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
                continue
            aligned += 1
            if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
                   for _tid, box in dets):
                colocated += 1
        return colocated / aligned if aligned else 0.0

```

```
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                 frame_width: float, frame_height: float,
198                                 court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                 net: Optional[Net] = None,
200                                 min_players: int = 2) -> Dict[str, float]:
201         """Aggregate the per-window person features from per-frame detections + the in-
window
202         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204         return {
205             "players_in_court": players_in_court(counts),
206             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208             "in_court_ratio": in_court_ratio(counts, min_players),
209             "shuttle_player_colocation": shuttle_player_colocation(
210                 per_frame, shuttle_points, frame_width, frame_height),
211         }
212
```

18. assistant (2026-06-13T19:15:42.453Z)

This is the critical confirmation. `shuttle_player_colocation` (line 178) does `per_frame.get(int(p.frame))` ? it directly looks up person detections by the **exact original-frame index** that `detections_per_frame` uses as its dict key. So the correctness of the frame-index keying in `detections_per_frame` (the `frames[frame_idx] = ...` with `frame_idx` being the absolute index) is load-bearing for alignment. If `detections_per_frame` keyed frames by a relative or off-by-one index, `shuttle_player_colocation` and `two_sided_motion` would silently produce wrong/zero values ? exactly the kind of bug that unit tests catch.

The `fusion_features.py` math is pure and has its own test (`test_fusion_features.py` exists). But that test feeds hand-built `per_frame` dicts; it does NOT prove `detections_per_frame` produces dicts keyed at the right absolute indices.

Let me confirm `test_fusion_features.py` doesn't somehow exercise `detections_per_frame`, and double-check the severity assessment by considering whether the loop logic is truly non-trivial.

19. user (2026-06-13T19:15:44.104Z)

No matches found

20. assistant (2026-06-13T19:15:59.560Z)

Confirmed ? `test_fusion_features.py` does not touch `detections_per_frame` or `VideoCapture` at all. It only tests the pure math.

Now let me do an adversarial check on the finding's specific testability claims to make sure the proposed test is actually achievable (otherwise the finding overstates how easy it is). The finding claims the harness in `_make_cap_mock` works. Two subtleties:

1. `_make_cap_mock`'s `mock_get_prop` returns `0.0` for `CAP_PROP_FRAME_HEIGHT` (it only special-cases FPS and FRAME_WIDTH). So `frame_height` would be `0.0` in a test using that exact helper. The finding's proposed assertion says "frame_width/frame_height/fps come from the cap" ?

frame_height from this mock is 0.0, but that's still a deterministic, assertable value. Minor.

2. `detections_per_frame` calls `cap.set(CAP_PROP_POS_FRAMES, start_frame)` then reads sequentially. The `_make_cap_mock` `read.side_effect` returns frames starting from index 0 regardless of the `set()` call (the mock doesn't model the seek). So in a test calling `detections_per_frame(start_frame=30, end_frame=40)`, the mock's `read` would still return `(True,"frame")` for each call. The loop sets `frame_idx=30`, reads, and keys `frames[30]`, `frames[33]`, `frames[36]`, `frames[39]` (with step=3). The mock returns enough frames. So the proposed assertion `{30,33,36,39}` is correct and achievable.

The finding's fix is accurate and achievable. Let me verify the modulo/boundary keying claim concretely: with `start_frame=30`, `end_frame=40`, `step=3`: frame_idx runs 30,31,...,40 (inclusive). `(frame_idx-30)%3==0` ? 30,33,36,39. 42 would be next but exceeds end_frame. So keys = {30,33,36,39}. Correct.

I have thoroughly verified all the claims. Let me assess severity. The finding labels it "high." Let me weigh this:

- It's a test-coverage gap, not a runtime bug. The code itself appears correct on reading.
- However, the gap is on genuinely bug-prone code (absolute seek + relative modulo + inclusive boundary), it's the ONLY new non-pure code, the alignment it produces is load-bearing for two downstream features (`shuttle_player_colocation`, `two_sided_motion`), and ? critically ? the existing fusion-hybrid test gives false confidence by mocking the whole method and only asserting `assert_called()`. The harness to test it already exists in the repo. This is a legitimate, actionable, non-nitpick finding.

The "high" severity is slightly aggressive for a test-coverage gap (no user-facing bug exists today), but it's well-justified as a real, actionable medium-to-high issue. I'll confirm `real=true` and note severity as medium-high ? the finding holds against the code.

21. user (2026-06-13T19:16:18.480Z)

Structured output provided successfully

22. assistant (2026-06-13T19:16:33.739Z)

Verified the finding against the actual code. It holds.

Summary of what I confirmed:
- `detections_per_frame` (`backend/pipeline/detectors/person_detector.py:256-301`) is entirely new in this change (git diff shows it added fresh after line 252) and is the only new non-pure code (the other edits ? `trajectory_hybrid.py` hooks, `config/models.py`, `fusion_features.py` ? are pure orchestration/schema/math).
- It has zero test coverage. The only references are the call site (`fusion_hybrid.py:98`) and `test_fusion_hybrid.py:100/109`, where the whole method is replaced with `fake_pd.detections_per_frame.return_value` and only `assert_called()` runs ? so the loop never executes. `test_fusion_features.py` doesn't touch it.
- The loop has exactly the bug-prone logic described: absolute seek (`cap.set(CAP_PROP_POS_FRAMES, start_frame)`), inclusive `while frame_idx <= end_frame`, stride `(frame_idx - start_frame) % step == 0` (modulo relative to start, not absolute index), plus the `fps<=0 -> 30.0` and not-opened fallbacks.
- The alignment it produces is load-bearing: `fusion_features.shuttle_player_colocation` does `per_frame.get(int(p.frame))` (`fusion_features.py:178`), a direct lookup by the exact absolute frame index the method emits as its dict key ? an off-by-one there would silently zero out colocation/two_sided_motion with no test failure.
- The proposed fix is achievable with the existing harness (`test_person_detector.py:36-57` already drives the same cv2-mock pattern).

I downgraded severity from "high" to medium: it's a test-coverage gap on correct-looking code (no present runtime bug), but it targets the single riskiest new path and the existing fusion test gives false confidence. One refinement for the fix author: `_make_cap_mock` returns `0.0` for `CAP_PROP_FRAME_HEIGHT` (only FPS/FRAME_WIDTH are special-cased), so either extend the mock or assert `frame_height == 0.0`.